



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

FACULTAD DE INGENIERÍA

Proyecto Integral de Ingeniería de Software

Tablero Kanban del Proyecto

Sistema Inteligente para la Gestión y Priorización de Incidencias de Soporte Técnico (SIGPI)

Versión 3 · adaptada a proyecto escolar · sprints 3 a 5 · React + TypeScript · FastAPI en capas

5 sprints (1 y 2 completados) · Scrum + RUP · 4 integrantes

EQUIPO

Haide Sánchez Gutiérrez · Laura Valeria Rodríguez Morales

Jesús Iniesta Valverde · Emmanuel Bernal Espejel

DOCENTE

Silvia Edith Albarrán Trujillo

METODOLOGÍA

Modelo de proceso: RUP (Proceso Unificado) | Seguimiento: Scrum

Toluca de Lerdo, Estado de México · octubre de 2026

Contenido

Nota de adaptación 3

1. Marco metodológico 4

2. Justificación de la metodología elegida 5

3. Arquitectura de referencia: React + TypeScript y FastAPI en capas 6

4. Leyenda y convenciones del tablero 7

5. Distribución de actividades por integrante 7

6. Tablero Kanban por sprint (sprints 3 a 5) 8

- Sprint 3 · RUP: Construcción I 8
- Sprint 4 · RUP: Construcción II 10
- Sprint 5 · RUP: Transición 12

7. Resumen de actividades por sprint 14

8. Trazabilidad: historias de usuario y tarjetas 15

9. Backlog posterior (entregas 2 y 3) 16

Referencias 17

Nota de adaptación

Esta versión adapta el tablero Kanban de la v2 a las condiciones de un proyecto escolar. Conserva sin cambios los sprints 1 y 2, ya completados, y replantea los sprints 3 a 5 (Construcción I, Construcción II y Transición): elimina las actividades que dependen de reuniones con el cliente y de servicios institucionales fuera del alcance del equipo, y reduce el esfuerzo a una carga compatible con cuatro estudiantes. El resto del documento (stack, arquitectura en capas y trazabilidad con las historias de usuario) se mantiene.

Aspecto	Tablero v2 (sprints 3 a 5)	Este tablero (v3)
Alcance de los sprints 3 a 5	71 tarjetas · 508 h (55 de desarrollo y 16 de gestión)	68 tarjetas · 304 h (53 de desarrollo y 15 de gestión)
Sprints 1 y 2	Planeados en la v2	Completados; se conservan sin cambios
Cliente	Sprint Review quincenal y pruebas de aceptación con el Jefe del DTIC-FI	Demostración del incremento a la docente (en clase, asesoría o video) y pruebas de aceptación realizadas por el equipo
Ceremonias Scrum	Planeación, revisión con el cliente y retrospectiva en tarjetas de 2 a 6 h	Tarjetas de gestión de 1 a 3 h, coordinadas desde el tablero; la retrospectiva se registra en una hoja compartida
Autenticación	Directorio institucional Microsoft 365 con inicio de sesión único	Inicio de sesión local con JWT y roles; directorio simulado con datos de prueba
Notificaciones	Correo institucional y aviso en la aplicación	Avisos solo dentro de la aplicación (campana de avisos); sin servicio de correo
Cobertura de pruebas	60 % al cierre del sprint 4 y 80 % al cierre del proyecto	60 % al cierre del proyecto
Horas por actividad	5 a 12 h por actividad de desarrollo (promedio de 8.3 h)	4 a 8 h por actividad de desarrollo (promedio de 5.3 h)
Reparto entre integrantes	Equilibrado entre desarrollo y gestión en todo el proyecto	Desarrollo: 69 a 72 h por persona; gestión: 5 a 7 h
Estado de las tarjetas	Todas en POR HACER	Sprints 3 a 5 en POR HACER; EN ELABORACIÓN y REALIZADO se completan durante cada sprint

Tabla 1. Cambios respecto del tablero anterior.
Elaboración propia.

Dependencias externas sustituidas. El directorio institucional se reemplaza por una tabla local con datos de prueba (si en el Sprint 2 ya se creó una equivalente, se reutiliza) y los avisos por correo por avisos dentro de la aplicación. Ambos quedan detrás de las capas *integrations/* y *services/*, de modo que pueden sustituirse por los servicios reales en una entrega posterior, tal como lo prevé la mitigación del riesgo RG-03 del Modelado de Negocios (autenticación e integraciones aisladas tras una capa de abstracción).

Criterios de aceptación por revisar en el Product Backlog: HU-01.1 y HU-01.3, cuyas tarjetas en la v2 dependían del directorio institucional, y HU-07.1 y HU-07.2, que dependían del correo. Su intención se conserva; cambia el medio con el que se cumplen.

Capacidad. El plan de los sprints 3 a 5 suma 304 horas durante seis semanas: ≈ 12.1, 12.5 y 13.4 h por integrante por semana en cada sprint. Si el equipo no dispone de esa capacidad, el recorte debe hacerse quitando historias completas y no actividades sueltas, empezando por las de menor dependencia con el resto del flujo (HU-03.3 y HU-07.2); las historias son la unidad de entrega.

1. Marco metodológico

El equipo del proyecto SIGPI adopta el Proceso Unificado (RUP) como modelo de proceso de desarrollo de software y Scrum como metodología de seguimiento. Son complementarias: RUP aporta la disciplina de artefactos, arquitectura y trazabilidad, y Scrum la cadencia y la transparencia necesarias para gestionar un equipo reducido de cuatro estudiantes con disponibilidad limitada.

Cada sprint de dos semanas se alinea con una fase del ciclo de vida de RUP. El Product Backlog se construye a partir de las historias de usuario, que derivan de los casos de uso del sistema y de la lista de dieciocho riesgos; cada historia se desglosa en actividades (tarjetas) con su entregable. Al tratarse de un proyecto escolar, no se cuenta con reuniones periódicas con el cliente institucional ni con sus servicios: la retroalimentación proviene de la docente, que revisa cada incremento al cierre del sprint, y las dependencias externas se simulan tras interfaces sustituibles.

¿Cómo se da seguimiento al proyecto? El avance se mide en sprints de duración fija (dos semanas); al cierre de cada uno se actualiza el tablero de la sección 6, se muestra el incremento a la docente y se registra la velocidad real (horas planeadas frente a ejecutadas) para replanificar. Este ciclo —planear, ejecutar, revisar, adaptar— se repite igual en los sprints restantes.

¿Cómo se identifica que se usa esta metodología? Tres señales la distinguen de un proceso en cascada o de un Scrum sin disciplina de artefactos: (1) artefactos formales de RUP —visión, arquitectura, matriz de trazabilidad— entregados de forma incremental; (2) un tablero con cadencia fija cada dos semanas; y (3) la gestión de riesgos integrada al backlog, con una tarjeta de revisión del RiskLog en cada sprint.

Dimensión	Scrum (seguimiento)	RUP (modelo de proceso)
Unidad de avance	Sprint de 2 semanas	Iteración dentro de una fase
Artefactos clave	Product Backlog, Sprint Backlog, Incremento	Visión, historias, arquitectura, Risklog
Roles	PO · SM · Developers	Analista · Diseñador · Programador · Tester
Gestión de riesgos	Ítem del backlog con criterio de aceptación	Planificación dirigida por riesgos en cada iteración
Trazabilidad	No prescrita (delegada al equipo)	Obligatoria: historia → diseño → código → prueba
Retroalimentación	Demostración del incremento al cierre de cada sprint	Hito de fase (Inicio, Elaboración, Construcción, Transición)

Tabla 2. Correspondencia entre las prácticas de Scrum y los artefactos y disciplinas de RUP adoptados por el equipo.
Elaboración propia.

2. Justificación de la metodología elegida

La elección se justifica confrontando las características del proyecto contra los cinco factores críticos que Boehm y Turner (2003) proponen para decidir el balance entre métodos ágiles y dirigidos por plan: tamaño, criticidad, dinamismo, personal y cultura.

Factor	Característica del proyecto	Zona ágil o disciplinada (Boehm y Turner, 2003)	Práctica adoptada
Tamaño	Equipo de 4 integrantes; alcance acotado a un semestre (5 sprints).	Los equipos pequeños se ubican en la zona ágil: la coordinación cara a cara sustituye a la documentación extensa.	Scrum aporta la cadencia ligera de seguimiento (Sprint Backlog, tablero).
Criticidad	Sistema de soporte interno; no compromete vidas ni pérdidas catastróficas, pero exige trazabilidad auditable al evaluarse como trabajo académico.	Una criticidad media no exige el rigor documental completo de un proceso dirigido por plan, pero no tolera la ausencia de artefactos formales.	RUP aporta la trazabilidad historia → código → prueba sin la sobrecarga de la cascada.
Dinamismo	El volumen de incidencias cambia y el clasificador no cuenta con datos históricos verificados (RT-01).	La alta incertidumbre de requisitos es el escenario que se asocia con la zona ágil.	Scrum permite replanificar el backlog cada sprint conforme se dispone de datos reales.
Personal	Cuatro estudiantes con experiencia en desarrollo pero sin rodaje como equipo Scrum.	Un equipo con experiencia técnica pero sin rodaje ágil se beneficia de plantillas y roles predefinidos.	Las plantillas de artefactos de RUP reducen la curva de aprendizaje.
Cultura	Sin acceso regular al cliente institucional; la docente valida los incrementos al cierre de cada sprint.	Una cultura de cliente distante favorece hitos de revisión espaciados y bien preparados.	La demostración al cierre de cada sprint sustituye a las reuniones con el cliente y coincide con un hito de fase.

Tabla 3. Justificación metodológica frente a los cinco factores críticos de selección.
Elaboración propia. Con base en Boehm y Turner (2003).

Conclusión. Ningún factor cae limpiamente en un extremo: el tamaño y el dinamismo empujan hacia la agilidad; la criticidad académica y la inexperiencia del equipo como unidad Scrum, hacia la disciplina de artefactos. Es la situación más común en la práctica y la que motiva un método híbrido: Scrum gobierna el ritmo y el seguimiento; RUP gobierna qué se documenta y cómo se demuestra la trazabilidad de cada requisito hasta su prueba.

3. Arquitectura de referencia: React + TypeScript y FastAPI en capas

Las tarjetas nombran su entregable en la capa que le corresponde. La regla de dependencia es única: cada capa solo conoce a la inmediatamente inferior, y el dominio no depende de ninguna. Así, las reglas de negocio (por ejemplo, la matriz de prioridad RN-05) se prueban sin base de datos ni servidor web.

3.1 Backend: FastAPI con arquitectura en capas

Capa	Paquete	Responsabilidad
Presentación (API)	api/routers · schemas · dependencias	Recibe la petición HTTP, valida con esquemas Pydantic, invoca un servicio y devuelve la respuesta. No contiene reglas de negocio.
Aplicación (servicios)	services/	Orquesta los casos de uso (registrar, clasificar, asignar, escalar) y aplica las reglas RN-xx. Es la única capa que coordina repositorios e integraciones.
Dominio	domain/	Reglas puras y estados: matriz urgencia-impacto, puntaje de idoneidad, reloj del SLA. Sin dependencias de framework.
Acceso a datos	repositories/ · models/ · alembic/	Modelos SQLAlchemy y repositorios que encapsulan las consultas; las migraciones versionan el esquema.
Integración	integrations/	Adaptadores aislados: directorio simulado (tabla local con datos de prueba) y clasificador. El servicio de correo y los servicios institucionales quedan fuera del alcance; la capa permite sustituirlos después.
Transversal	core/	Configuración por entorno, seguridad (JWT y roles), excepciones, registro de eventos y auditoría.

Tabla 4. Capas del backend y su responsabilidad.
Elaboración propia.

3.2 Frontend: React + TypeScript por features

Elemento	Responsabilidad
app / routes	Enrutamiento (React Router), proveedores globales y protección de rutas por rol.
features/<dominio>	Un directorio por dominio (solicitudes, clasificacion, bandeja, sla, catalogos, usuarios, notificaciones) con sus páginas, componentes y hooks; es la unidad de trabajo de las tarjetas de interfaz.
api/	Cliente tipado; los tipos TypeScript se generan del esquema OpenAPI de FastAPI para que el contrato no se duplique a mano.
Estado de servidor	TanStack Query para consultas y mutaciones; TanStack Table para bandejas y listados.
shared/ui	Componentes reutilizables (shadcn/ui) y utilidades sin dependencia de un dominio.
Pruebas	Vitest con React Testing Library para componentes y hooks; TypeScript estricto y ESLint en el pipeline CI.

Tabla 5. Organización del frontend.
Elaboración propia.

3.3 Pruebas y calidad

El backend usa pytest con httpx y una base de datos de prueba; la cobertura mínima es del 60 % al cierre del proyecto. El pipeline CI (construido en el Sprint 2) ejecuta ruff, pytest, el compilador de TypeScript, ESLint y Vitest en cada solicitud de cambio, y ninguna tarjeta de desarrollo se da por terminada sin su prueba asociada.

4. Leyenda y convenciones del tablero

HS — Haide Sánchez · LR — Laura Rodríguez · JI — Jesús Iniesta · EB — Emmanuel Bernal. Cada tarjeta lleva un identificador (Sn-nn), su descripción, la categoría, el responsable, las horas, las historias de las que deriva y su entregable (→). Las tarjetas con borde azul son de **Desarrollo** (diseño, código, pruebas, documentación); las de borde ámbar son de **Gestión** (planeación, riesgos, trazabilidad, demostración y retrospectiva).

Tipo	Horas	Alcance
D — Diseño	4 h	Modelos, contratos y reglas antes de codificar
B — Servidor	6 h	Routers, servicios, repositorios y esquemas
F — Interfaz	6 h	Páginas, componentes y hooks en React + TypeScript
I — Integración	5 h	Pasos que conectan capas y adaptadores simulados
M — Datos y modelo	8 h	Conjunto de datos y entrenamiento del clasificador
P — Pruebas	4 h	Pruebas automatizadas de los criterios de aceptación
X — Documentación	4 h	Despliegue y manual
G — Gestión	1 a 3 h	Planeación, riesgos, trazabilidad, demostración y retrospectiva

Tabla 6. Horas planeadas por tipo de actividad (sprints 3 a 5).
Elaboración propia. Valores reducidos respecto de la v2 según la capacidad del equipo; son horas planeadas, no medidas.

5. Distribución de actividades por integrante

Todos los integrantes participan tanto en desarrollo como en gestión. La asignación se hizo balanceando las horas de cada categoría y las horas de servidor, interfaz y pruebas de cada persona; ningún integrante concentra un solo tipo de actividad. Las tablas 7 y 8 corresponden a los sprints 3 a 5; la tabla 9 incluye también los sprints 1 y 2 (completados).

Integrante	Gestión	Desarrollo	Total
Haide Sánchez	3 tareas · 5 h	12 tareas · 71 h	15 tareas · 76 h
Laura Rodríguez	3 tareas · 6 h	13 tareas · 69 h	16 tareas · 75 h
Jesús Iniesta	3 tareas · 5 h	14 tareas · 72 h	17 tareas · 77 h
Emmanuel Bernal	6 tareas · 7 h	14 tareas · 69 h	20 tareas · 76 h
Total	**15 tareas · 23 h**	**53 tareas · 281 h**	**68 tareas · 304 h**

Tabla 7. Distribución de actividades por integrante, por categoría y carga horaria (sprints 3 a 5).
Elaboración propia.

Integrante	Servidor	Interfaz	Pruebas	Integración	Datos, diseño y otros
Haide Sánchez	30	12	8	5	16
Laura Rodríguez	24	24	16	5	0
Jesús Iniesta	30	18	16	0	8
Emmanuel Bernal	24	12	20	5	8

Tabla 8. Horas de desarrollo por integrante y por tipo de actividad (sprints 3 a 5).
Elaboración propia.

Integrante	Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5
Haide Sánchez	24	36	24	25	27
Laura Rodríguez	27	32	24	25	26
Jesús Iniesta	16	45	25	25	27
Emmanuel Bernal	16	41	24	25	27

Tabla 9. Horas por integrante y por sprint (sprints 1 y 2 completados).
Elaboración propia.

La diferencia entre la persona con más y con menos horas es de 3 h en desarrollo (4 % del promedio) y de 2 h en gestión. Los roles de Scrum (Haide como Product Owner y Emmanuel como Scrum Master) no reducen su carga de programación: la gestión de planeación, riesgos y demostración está repartida por tarjetas y no concentrada en los roles.

6. Tablero Kanban por sprint

Los sprints 1 y 2 están completados (14 y 21 tarjetas, 83 y 154 h planeadas en la v2) y no se vuelven a listar. Las tarjetas de los sprints 3 a 5 están en POR HACER porque el tablero es el plan; las columnas EN ELABORACIÓN y REALIZADO se completan durante la ejecución, con el límite de trabajo en curso que acuerde el equipo.

Sprint 3 · RUP: Construcción I

22 tarjetas · 97 h (Desarrollo 91 h, Gestión 6 h) · ≈ 12.1 h por integrante por semana · Historias: HU-01.1, HU-01.3, HU-02.3, HU-04.1, HU-04.2

POR HACER (22)	EN ELABORACIÓN (0)	REALIZADO (0)
S3-01 · Manejo de errores y registro de eventos Excepciones de dominio, manejador global en FastAPI y formato uniforme de error; registro estructurado de eventos. Desarrollo (servidor) · Laura Rodríguez · 6 h → core/exceptions.py · core/logging.py	*Se actualiza durante el sprint*	*Se actualiza durante el sprint*
S3-02 · Formulario de registro de solicitud Formulario de captura con validación en línea y acuse de folio. Desarrollo (interfaz) · Emmanuel Bernal · 6 h · HU-01.1 → features/solicitudes/components/SolicitudForm.tsx · hooks/useCrearSolicitud.ts		
S3-03 · Inicio de sesión con JWT y roles Autenticación local: contraseñas con hash, emisión de token JWT y verificación del rol en cada ruta. Sustituye al directorio institucional. Desarrollo (servidor) · Haide Sánchez · 6 h · HU-01.1 → api/routers/auth.py · core/security.py		
S3-04 · Pantalla de inicio de sesión y rutas protegidas Formulario de acceso, almacenamiento del token y protección de rutas por rol en React Router. Desarrollo (interfaz) · Jesús Iniesta · 6 h · HU-01.1 → features/auth/pages/LoginPage.tsx · app/routes		
S3-05 · Pruebas del alta de solicitud Automatizar los tres criterios: alta válida, campos incompletos y credenciales inválidas. Desarrollo (pruebas) · Jesús Iniesta · 4 h · HU-01.1 → tests/api/test_solicitudes.py · SolicitudForm.test.tsx		
S3-06 · Servicio de cálculo de prioridad Calcular la prioridad de forma centralizada con la matriz urgencia-impacto y bloquear su edición directa. Desarrollo (servidor) · Emmanuel Bernal · 6 h · HU-02.3 → services/prioridad_service.py		
S3-07 · Pruebas de las nueve combinaciones de la matriz Probar las nueve combinaciones de la matriz y el intento de edición directa. Desarrollo (pruebas) · Jesús Iniesta · 4 h · HU-02.3 → tests/domain/test_priority_matrix.py		
S3-08 · Endpoint de registro asistido Habilitar el registro asistido con canal de origen (teléfono o presencial). Desarrollo (servidor) · Haide Sánchez · 6 h · HU-01.3 → api/routers/solicitudes.py (POST /asistido) · services/solicitud_service.py		
S3-09 · Pantalla de registro asistido Agregar la variante de captura para el técnico con búsqueda del solicitante. Desarrollo (interfaz) · Laura Rodríguez · 6 h · HU-01.3 → features/solicitudes/pages/RegistroAsistidoPage.tsx		
S3-10 · Solicitante no localizado: marca «por verificar» Consultar el directorio simulado (tabla local con datos de prueba) y, si el solicitante no aparece, registrar la solicitud con la marca «por verificar». Sustituye al directorio institucional. Desarrollo (integración) · Haide Sánchez · 5 h · HU-01.3 → integrations/directorio_simulado.py · domain/estados.py		

POR HACER (22)	EN ELABORACIÓN (0)	REALIZADO (0)
S3-11 · Pruebas del registro asistido y la contingencia Probar el registro asistido y el caso de solicitante no localizado. Desarrollo (pruebas) · Laura Rodríguez · 4 h · HU-01.3 → tests/api/test_registro_asistido.py		
S3-12 · Servicio de seguimiento y resolución Registrar diagnóstico, solución y cambios de estatus; calcular el tiempo de resolución. Desarrollo (servidor) · Haide Sánchez · 6 h · HU-04.1 → api/routers/seguimiento.py · services/seguimiento_service.py		
S3-13 · Detalle de la solicitud con su bitácora Construir el detalle de la solicitud con su bitácora. Desarrollo (interfaz) · Laura Rodríguez · 6 h · HU-04.1 → features/solicitudes/pages/SolicitudDetallePage.tsx		
S3-14 · Pruebas de resolución y confirmación Probar la resolución y la solicitud de confirmación. Desarrollo (pruebas) · Jesús Iniesta · 4 h · HU-04.1 → tests/services/test_seguimiento.py		
S3-15 · Estado «En espera» y reloj del SLA Implementar el estado «En espera» y la suspensión y reanudación del reloj del SLA. Desarrollo (servidor) · Emmanuel Bernal · 6 h · HU-04.2 → services/sla_clock_service.py · domain/estados.py		
S3-16 · Motivo de espera en la pantalla de atención Agregar el motivo de espera en la pantalla de atención. Desarrollo (interfaz) · Jesús Iniesta · 6 h · HU-04.2 → features/solicitudes/components/EsperaDialog.tsx		
S3-17 · Pruebas de suspensión y reanudación del reloj Probar el descuento del periodo suspendido. Desarrollo (pruebas) · Emmanuel Bernal · 4 h · HU-04.2 → tests/services/test_sla_clock.py		
S3-18 · Planeación del Sprint 3 Confirmar el alcance, repartir las tarjetas y reservar el 20 % del tiempo para imprevistos. Gestión · Emmanuel Bernal · 1 h → Sprint Backlog 3		
S3-19 · Estrategia de ramas y revisión de pull requests Definir una rama por tarjeta, la revisión de cada pull request por un compañero y el criterio de fusión. Gestión · Laura Rodríguez · 2 h → Guía de contribución		
S3-20 · Revisión de riesgos del sprint 3 Actualizar el RiskLog con la información del incremento. Gestión · Haide Sánchez · 1 h → RiskLog v3		
S3-21 · Demostración del incremento (Sprint 3) Presentar el flujo de alta y seguimiento a la docente, en clase o asesoría, y anotar la retroalimentación; si no hay sesión, entregar un video corto de la demostración. Gestión · Emmanuel Bernal · 1 h → Notas de retroalimentación		
S3-22 · Retrospectiva 3 y velocidad Comparar horas planeadas y reales en una hoja compartida y replanificar el sprint 4. Gestión · Jesús Iniesta · 1 h → Registro de velocidad		

Cómo ejecutar el Sprint 3 (RUP: Construcción I)

1. Primer incremento funcional: el flujo de alta con inicio de sesión local (HU-01.1), el registro asistido (HU-01.3) y el seguimiento con bitácora (HU-04.1) operan de punta a punta.

2. Calcular la prioridad con la matriz urgencia-impacto (HU-02.3) y suspender el reloj del SLA (HU-04.2).

3. Reservar el 20 % del tiempo del sprint para imprevistos; aplicar el flujo de ramas y la revisión de pull requests entre compañeros.

4. Demostrar el incremento a la docente (en clase, asesoría o video) y registrar la retroalimentación.

5. Retrospectiva: comparar velocidad real y estimada y replanificar el sprint 4.

Sprint 4 · RUP: Construcción II

22 tarjetas · 100 h (Desarrollo 93 h, Gestión 7 h) · ≈ 12.5 h por integrante por semana · Historias: HU-02.1, HU-02.2, HU-04.3, HU-07.1, HU-07.2

POR HACER (22)	EN ELABORACIÓN (0)	REALIZADO (0)
S4-01 · Consulta por folio y confirmación de la solución Exponer la consulta por folio y la confirmación de la solución por el usuario. Desarrollo (servidor) · Jesús Iniesta · 6 h · HU-04.3 → api/routers/solicitudes.py (GET /{folio}, POST /confirmar)	*Se actualiza durante el sprint*	*Se actualiza durante el sprint*
S4-02 · Vista de estatus para el usuario Construir la vista de estatus para el usuario. Desarrollo (interfaz) · Jesús Iniesta · 6 h · HU-04.3 → features/solicitudes/pages/MiSolicitudPage.tsx		
S4-03 · Pruebas de consulta y cierre por confirmación Probar la consulta y el cierre por confirmación. Desarrollo (pruebas) · Haide Sánchez · 4 h · HU-04.3 → tests/api/test_consulta_folio.py		
S4-04 · Catálogo de eventos y plantillas de aviso Definir los eventos que generan aviso, sus destinatarios y el texto de cada aviso. Desarrollo (diseño) · Emmanuel Bernal · 4 h · HU-07.1 → domain/eventos.py · templates/avisos/		
S4-05 · Servicio de notificaciones dentro de la aplicación Generar un aviso por evento con folio, prioridad, técnico y fecha compromiso, y guardarlo para cada destinatario. Sustituye al correo institucional. Desarrollo (servidor) · Laura Rodríguez · 6 h · HU-07.1 → services/notificacion_service.py · repositories/notificacion_repository.py		
S4-06 · Centro de notificaciones en la interfaz Campana con contador de avisos no leídos y lista de avisos, con consulta periódica al servidor. Desarrollo (interfaz) · Haide Sánchez · 6 h · HU-07.1 → features/notificaciones/components/CampanaAvisos.tsx		
S4-07 · Pruebas de emisión de avisos Probar que cada evento genera su aviso para los destinatarios correctos y que se muestra en menos de un minuto. Desarrollo (pruebas) · Emmanuel Bernal · 4 h · HU-07.1 → tests/services/test_notificacion.py		
S4-08 · Monitor del 80 % del tiempo comprometido Detectar, con una tarea programada, las solicitudes que alcanzan el 80 % del tiempo comprometido. Desarrollo (servidor) · Jesús Iniesta · 6 h · HU-07.2 → services/sla_monitor_service.py · tarea programada		
S4-09 · Alertas de riesgo al técnico y al Jefe Emitir la alerta al técnico y al Jefe del Departamento como aviso de alta importancia dentro de la aplicación. Desarrollo (servidor) · Jesús Iniesta · 6 h · HU-07.2 → services/notificacion_service.py (alerta_riesgo)		

POR HACER (22)	EN ELABORACIÓN (0)	REALIZADO (0)
S4-10 · Pruebas del umbral de alerta Probar la desviación máxima de un minuto. Desarrollo (pruebas) · Emmanuel Bernal · 4 h · HU-07.2 → tests/services/test_sla_monitor.py		
S4-11 · Conjunto de datos etiquetado Construir y etiquetar el conjunto de entrenamiento con solicitudes redactadas por el equipo con el vocabulario del DTIC, balanceado por categoría. Desarrollo (datos y modelo) · Haide Sánchez · 8 h · HU-02.1 → ml/etl/build_dataset.py · ml/data/dataset_v1.csv		
S4-12 · Servicio de clasificación (reglas y modelo) Exponer el clasificador como servicio y guardar versión y confianza. Desarrollo (servidor) · Emmanuel Bernal · 6 h · HU-02.1 → services/clasificacion_service.py · integrations/clasificador_client.py		
S4-13 · Integración de la clasificación en el alta Invocar el clasificador al registrar y pasar el estatus a «Clasificada». Desarrollo (integración) · Emmanuel Bernal · 5 h · HU-02.1 → services/solicitud_service.py (paso de clasificación)		
S4-14 · Umbral de confianza y cola de revisión Definir el umbral configurable (0.75 por defecto) y enviar a la cola de revisión las predicciones por debajo de él. Desarrollo (servidor) · Haide Sánchez · 6 h · HU-02.2 → core/config.py · services/revision_service.py · repositories/revision_repository.py		
S4-15 · Pantalla de revisión de clasificaciones Construir la pantalla de revisión con edición y justificación obligatoria. Desarrollo (interfaz) · Laura Rodríguez · 6 h · HU-02.2 → features/clasificacion/pages/RevisionPage.tsx		
S4-16 · Registro de correcciones como ejemplos Guardar cada corrección como ejemplo para el reentrenamiento. Desarrollo (servidor) · Laura Rodríguez · 6 h · HU-02.2 → repositories/correccion_repository.py		
S4-17 · Pruebas de revisión y corrección Probar predicción incierta y corrección con justificación. Desarrollo (pruebas) · Laura Rodríguez · 4 h · HU-02.2 → tests/api/test_revision.py		
S4-18 · Planeación del Sprint 4 Confirmar el alcance y ordenar las dependencias de notificaciones y clasificación. Gestión · Emmanuel Bernal · 1 h → Sprint Backlog 4		
S4-19 · Matriz de trazabilidad historia → código → prueba Vincular cada historia con su módulo, su prueba y su criterio de aceptación. Gestión · Laura Rodríguez · 3 h → Trazabilidad.xlsx		
S4-20 · Revisión de riesgos del sprint 4 Revisar la exactitud del clasificador y la disponibilidad de datos (RT-01). Gestión · Haide Sánchez · 1 h → RiskLog v4		
S4-21 · Demostración del incremento (Sprint 4) Presentar notificaciones y clasificación con revisión humana a la docente y anotar la retroalimentación. Gestión · Jesús Iniesta · 1 h → Notas de retroalimentación		
S4-22 · Retrospectiva 4 y velocidad Evaluar la velocidad y ajustar el último sprint. Gestión · Emmanuel Bernal · 1 h → Registro de velocidad		

Cómo ejecutar el Sprint 4 (RUP: Construcción II)

1. Segundo incremento: consulta de estatus para el usuario (HU-04.3), avisos dentro de la aplicación (HU-07.1) y alertas de riesgo de incumplimiento (HU-07.2).

2. Clasificación con revisión humana: conjunto de datos, servicio de clasificación e integración (HU-02.1), umbral y cola de revisión (HU-02.2).

3. Iniciar la recolección de datos etiquetados para entrenar el modelo y mitigar RT-01; el clasificador arranca con reglas explícitas.

4. Construir la matriz de trazabilidad historia → código → prueba.

5. Demostrar a la docente las notificaciones y la clasificación con revisión humana.

Sprint 5 · RUP: Transición

24 tarjetas · 107 h (Desarrollo 97 h, Gestión 10 h) · ≈ 13.4 h por integrante por semana · Historias: HU-02.1, HU-02.5, HU-03.1, HU-03.3, HU-04.6

POR HACER (24)	EN ELABORACIÓN (0)	REALIZADO (0)
S5-01 · Consulta de la bandeja del técnico Consultar las solicitudes asignadas con orden por prioridad y tiempo restante. Desarrollo (servidor) · Laura Rodríguez · 6 h · HU-04.6 → api/routers/bandeja.py · repositories/solicitud_repository.py (filtros)	*Se actualiza durante el sprint*	*Se actualiza durante el sprint*
S5-02 · Tabla de bandeja con filtros y paginación Construir la tabla paginada con filtros por estatus, prioridad y técnico. Desarrollo (interfaz) · Haide Sánchez · 6 h · HU-04.6 → features/bandeja/pages/BandejaPage.tsx (TanStack Table)		
S5-03 · Pruebas de orden, filtros y paginación Probar orden, filtros y paginación con un conjunto de datos de prueba amplio. Desarrollo (pruebas) · Emmanuel Bernal · 4 h · HU-04.6 → tests/api/test_bandeja.py		
S5-04 · Consulta de la cola de revisión Consultar las solicitudes pendientes de revisión por antigüedad y prioridad. Desarrollo (servidor) · Jesús Iniesta · 6 h · HU-02.5 → api/routers/revision.py (GET /cola)		
S5-05 · Vista de la cola de revisión Construir la vista de la cola de revisión. Desarrollo (interfaz) · Emmanuel Bernal · 6 h · HU-02.5 → features/clasificacion/pages/ColaRevisionPage.tsx		
S5-06 · Pruebas de orden y estado vacío Probar orden y estado vacío. Desarrollo (pruebas) · Jesús Iniesta · 4 h · HU-02.5 → tests/api/test_cola_revision.py		
S5-07 · Entrenamiento y evaluación del clasificador Entrenar y evaluar el clasificador de categoría, urgencia e impacto; si los datos son insuficientes, conservar las reglas explícitas. Desarrollo (datos y modelo) · Haide Sánchez · 8 h · HU-02.1 → ml/training/train_classifier.py · ml/models/clasificador_v1.joblib		
S5-08 · Pruebas de desempeño de la clasificación Medir el tiempo de respuesta (menos de cinco segundos) y probar los criterios. Desarrollo (pruebas) · Haide Sánchez · 4 h · HU-02.1 → tests/services/test_clasificacion.py		
S5-09 · Fórmula del puntaje de idoneidad Definir la fórmula del puntaje de idoneidad: especialidad, disponibilidad y carga. Desarrollo (diseño) · Jesús Iniesta · 4 h · HU-03.1 → domain/assignment_scoring.py		
S5-10 · Servicio de asignación automática Seleccionar al candidato de mayor puntaje entre los técnicos elegibles. Desarrollo (servidor) · Emmanuel Bernal · 6 h · HU-03.1 → api/routers/asignacion.py · services/asignacion_service.py		

POR HACER (24)	EN ELABORACIÓN (0)	REALIZADO (0)
S5-11 · Cambio de estatus y aviso al técnico Pasar el estatus a «Asignada» y notificar al técnico dentro de la aplicación. Desarrollo (integración) · Laura Rodríguez · 5 h · HU-03.1 → services/asignacion_service.py → notificacion_service		
S5-12 · Desglose del puntaje en la interfaz Mostrar el desglose del puntaje del técnico asignado. Desarrollo (interfaz) · Laura Rodríguez · 6 h · HU-03.1 → features/solicitudes/components/PuntajeDesglose.tsx		
S5-13 · Pruebas de asignación por carga y especialidad Probar la asignación con distintas cargas y especialidades. Desarrollo (pruebas) · Laura Rodríguez · 4 h · HU-03.1 → tests/domain/test_assignment_scoring.py		
S5-14 · Regla de carga máxima y prioridad P1 Ignorar la carga máxima en P1 y encolar en la cola general las demás. Desarrollo (servidor) · Haide Sánchez · 6 h · HU-03.3 → domain/assignment_rules.py		
S5-15 · Temporizador de cinco minutos para P1 y alerta al Jefe Programar la alerta a los cinco minutos sin asignar una solicitud P1 y enviarla al Jefe del Departamento como aviso de alta importancia dentro de la aplicación. Desarrollo (servidor) · Jesús Iniesta · 6 h · HU-03.3 → services/asignacion_service.py (planificador) · services/notificacion_service.py		
S5-16 · Pruebas de los tres escenarios de carga Probar los tres escenarios de carga. Desarrollo (pruebas) · Laura Rodríguez · 4 h · HU-03.3 → tests/services/test_asignacion_p1.py		
S5-17 · Ampliar la cobertura de pruebas al 60 % Completar pruebas de servicios y dominio hasta el umbral de cobertura mínimo para el cierre. Desarrollo (pruebas) · Emmanuel Bernal · 4 h → Informe de cobertura		
S5-18 · Documentación de despliegue README con requisitos, variables de entorno, Docker Compose y respaldo de la base de datos. Desarrollo (documentación) · Emmanuel Bernal · 4 h → DEPLOY.md		
S5-19 · Manual de usuario Guía breve para el solicitante, el técnico, el Jefe del Departamento y el administrador. Desarrollo (documentación) · Jesús Iniesta · 4 h → ManualUsuario.pdf		
S5-20 · Planeación del Sprint 5 y lista de verificación de entrega Confirmar el alcance final y preparar la lista de verificación de entrega. Gestión · Emmanuel Bernal · 1 h → Sprint Backlog 5		
S5-21 · Pruebas de aceptación del equipo Ejecutar los escenarios de aceptación de las historias en un ambiente de prueba y registrar defectos y correcciones. Gestión · Jesús Iniesta · 3 h → Registro de aceptación		
S5-22 · Presentación final y demostración en vivo Demostración del sistema ante la docente, lecciones aprendidas y mapa de cobertura de historias. Gestión · Haide Sánchez · 3 h → Presentación final		

POR HACER (24)	EN ELABORACIÓN (0)	REALIZADO (0)
S5-23 · Retrospectiva del proyecto y métricas Consolidar velocidad, cobertura de historias y lecciones aprendidas. Gestión · Emmanuel Bernal · 2 h → Informe de cierre		
S5-24 · Actualizar el estado del backlog posterior Documentar qué historias de las entregas 2 y 3 quedan pendientes y su prioridad. Gestión · Laura Rodríguez · 1 h → Product Backlog v5		

Cómo ejecutar el Sprint 5 (RUP: Transición)

1. Cerrar la primera entrega: bandeja del técnico (HU-04.6), cola de revisión (HU-02.5), asignación automática (HU-03.1) y regla de solicitudes críticas (HU-03.3).
2. Entrenar y evaluar el modelo (HU-02.1); si los datos son insuficientes, documentar el plan y mantener las reglas.
3. Ampliar la cobertura de pruebas al 60 % y producir la documentación de despliegue y el manual de usuario.
4. Pruebas de aceptación del equipo y demostración final ante la docente.
5. Retrospectiva del proyecto y actualización del backlog posterior con las historias de las entregas 2 y 3.

7. Resumen de actividades por sprint

Sprint	Fase RUP	Estado	Desarrollo	Gestión	Total tarjetas	Horas
1	RUP: Inicio	Completado	10	4	14	83
2	RUP: Elaboración	Completado	17	4	21	154
3	RUP: Construcción I	Por hacer	17	5	22	97
4	RUP: Construcción II	Por hacer	17	5	22	100
5	RUP: Transición	Por hacer	19	5	24	107
Sprints 3 a 5			**53**	**15**	**68**	**304**
Proyecto (1 a 5)			**80**	**23**	**103**	**541**

Tabla 10. Resumen de actividades de Desarrollo y de Gestión por sprint.
Elaboración propia.

8. Trazabilidad: historias de usuario y tarjetas

Cada historia se vincula con las tarjetas que la implementan. Las de prioridad «Debe» están programadas en los sprints (las tarjetas S1 y S2, en gris, ya están completadas); las demás pasan al backlog posterior (sección 9).

Historia	Título	Actor	Prioridad	Sprint	Tarjetas
HU-01.1	Registrar una solicitud desde el portal	ACT-01	Debe	1, 2, 3	S1-09, S2-16, S3-02, S3-03, S3-04, S3-05
HU-01.2	Indicar el equipo afectado y adjuntar evidencia	ACT-01	Debería	Posterior	—
HU-01.3	Registrar una solicitud recibida por teléfono o en persona	ACT-02	Debe	3	S3-08, S3-09, S3-10, S3-11
HU-01.4	Evitar solicitudes duplicadas	ACT-01	Debería	Posterior	—
HU-01.5	Registrar una solicitud correctiva durante un mantenimiento	ACT-03	Debería	Posterior	—
HU-02.1	Proponer categoría, urgencia e impacto automáticamente	ACT-02	Debe	4, 5	S4-11, S4-12, S4-13, S5-07, S5-08
HU-02.2	Revisar y corregir clasificaciones de baja confianza	ACT-02	Debe	4	S4-14, S4-15, S4-16, S4-17
HU-02.3	Calcular la prioridad con la matriz urgencia-impacto	ACT-04	Debe	1, 3	S1-10, S3-06, S3-07
HU-02.4	Continuar el servicio si el motor no responde	ACT-02	Debería	Posterior	—
HU-02.5	Consultar la cola de revisión de clasificaciones	ACT-02	Debe	5	S5-04, S5-05, S5-06
HU-03.1	Proponer al técnico idóneo	ACT-04	Debe	5	S5-09, S5-10, S5-11, S5-12, S5-13
HU-03.2	Sustituir manualmente al técnico propuesto	ACT-02	Debería	Posterior	—
HU-03.3	Garantizar la asignación de solicitudes críticas	ACT-04	Debe	5	S5-14, S5-15, S5-16
HU-04.1	Registrar el avance en una bitácora única	ACT-02 · ACT-03	Debe	2, 3	S2-17, S3-12, S3-13, S3-14
HU-04.2	Suspender el reloj del SLA por causas ajenas	ACT-02 · ACT-03	Debe	3	S3-15, S3-16, S3-17
HU-04.3	Consultar el estatus y confirmar la solución	ACT-01	Debe	4	S4-01, S4-02, S4-03
HU-04.4	Reabrir una solicitud dentro del plazo	ACT-01	Debería	Posterior	—
HU-04.5	Cerrar automáticamente las solicitudes sin respuesta	ACT-04	Debería	Posterior	—
HU-04.6	Consultar la bandeja de solicitudes asignadas	ACT-02 · ACT-03	Debe	5	S5-01, S5-02, S5-03
HU-05.1	Solicitar un escalamiento con evidencia	ACT-03	Debería	Posterior	—
HU-05.2	Autorizar o rechazar el escalamiento	ACT-04	Debería	Posterior	—
HU-05.3	Escalar automáticamente solicitudes críticas sin respuesta	ACT-04	Podría	Posterior	—
HU-05.4	Recibir la solicitud escalada con su historial	ACT-03	Debería	Posterior	—
HU-06.1	Buscar soluciones antes de diagnosticar	ACT-02 · ACT-03	Debería	Posterior	—
HU-06.2	Documentar una solución nueva	ACT-03	Debería	Posterior	—
HU-06.3	Consultar artículos de autoayuda	ACT-01	Podría	Posterior	—
HU-07.1	Recibir avisos del ciclo de vida	ACT-01	Debe	4	S4-04, S4-05, S4-06, S4-07
HU-07.2	Alertar el riesgo de incumplimiento	ACT-02 · ACT-03	Debe	4	S4-08, S4-09, S4-10
HU-07.3	No perder avisos si falla el correo	ACT-01	Debería	Posterior	—

Historia	Título	Actor	Prioridad	Sprint	Tarjetas
HU-08.1	Definir y versionar los acuerdos de nivel de servicio	ACT-04	Debe	1, 2	S1-07, S2-13, S2-14, S2-15
HU-09.1	Ver los indicadores del servicio	ACT-04	Debería	Posterior	—
HU-09.2	Exportar los indicadores	ACT-04	Debería	Posterior	—
HU-10.1	Administrar cuentas y roles	ACT-09	Debe	1, 2	S1-06, S2-04, S2-05, S2-06
HU-10.2	Proteger la continuidad de la administración	ACT-09	Debería	Posterior	—
HU-10.3	Auditar las acciones de administración	ACT-09	Debe	2	S2-07, S2-08, S2-09
HU-11.1	Mantener el catálogo de técnicos y áreas	ACT-09	Debe	1, 2	S1-08, S2-10, S2-11, S2-12
HU-11.2	Evitar categorías sin cobertura	ACT-09	Debería	Posterior	—
HU-11.3	Mantener el inventario de activos y el calendario de mantenimiento	ACT-04	Debería	Posterior	—
HU-11.4	Actualizar la ficha del activo y emitir el acta de mantenimiento	ACT-03	Debería	Posterior	—
HU-12.1	Supervisar la exactitud del clasificador	ACT-09	Podría	Posterior	—
HU-12.2	Reentrenar el modelo con las solicitudes verificadas	ACT-09	Podría	Posterior	—

Tabla 11. Matriz de trazabilidad entre las historias de usuario y las tarjetas del tablero.
Elaboración propia. Con base en las Historias de usuario, versión 3.

9. Backlog posterior (entregas 2 y 3)

Las 23 historias de prioridad «Debería» y «Podría» conservan sus 76 actividades, sin responsable ni sprint asignado; el detalle de cada actividad se mantiene sin cambios en la sección 9 del tablero v2. Se planean en cuanto se cierre la primera entrega y, para asignarlas, se aplicará el mismo criterio de equidad entre desarrollo y gestión. La última columna señala las historias que incluyen actividades que dependen de un servicio externo; mientras no se cuente con él, deben resolverse con datos locales o diferirse.

Historia	Título	Prioridad	Entrega	Actividades	Servicio externo
HU-01.2	Indicar el equipo afectado y adjuntar evidencia	Debería	Entrega 2	4	—
HU-01.4	Evitar solicitudes duplicadas	Debería	Entrega 2	4	—
HU-01.5	Registrar una solicitud correctiva durante un mantenimiento	Debería	Entrega 2	4	—
HU-02.4	Continuar el servicio si el motor no responde	Debería	Entrega 2	3	—
HU-03.2	Sustituir manualmente al técnico propuesto	Debería	Entrega 2	3	—
HU-04.4	Reabrir una solicitud dentro del plazo	Debería	Entrega 2	3	—
HU-04.5	Cerrar automáticamente las solicitudes sin respuesta	Debería	Entrega 2	2	—
HU-05.1	Solicitar un escalamiento con evidencia	Debería	Entrega 2	4	—
HU-05.2	Autorizar o rechazar el escalamiento	Debería	Entrega 2	4	Registro ante la DTIC Central o el proveedor (1 actividad)
HU-05.3	Escalar automáticamente solicitudes críticas sin respuesta	Podría	Entrega 3	2	—
HU-05.4	Recibir la solicitud escalada con su historial	Debería	Entrega 2	2	—
HU-06.1	Buscar soluciones antes de diagnosticar	Debería	Entrega 2	4	—
HU-06.2	Documentar una solución nueva	Debería	Entrega 2	4	—
HU-06.3	Consultar artículos de autoayuda	Podría	Entrega 3	3	—
HU-07.3	No perder avisos si falla el correo	Debería	Entrega 2	2	Servicio de correo (2 actividades)
HU-09.1	Ver los indicadores del servicio	Debería	Entrega 2	4	—
HU-09.2	Exportar los indicadores	Debería	Entrega 2	3	—

Historia	Título	Prioridad	Entrega	Actividades	Servicio externo
HU-10.2	Proteger la continuidad de la administración	Debería	Entrega 2	3	—
HU-11.2	Evitar categorías sin cobertura	Debería	Entrega 2	2	—
HU-11.3	Mantener el inventario de activos y el calendario de mantenimiento	Debería	Entrega 2	5	Repositorio patrimonial (1 actividad)
HU-11.4	Actualizar la ficha del activo y emitir el acta de mantenimiento	Debería	Entrega 2	4	—
HU-12.1	Supervisar la exactitud del clasificador	Podría	Entrega 3	3	—
HU-12.2	Reentrenar el modelo con las solicitudes verificadas	Podría	Entrega 3	4	—

Tabla 12. Historias del backlog posterior y número de actividades.
Elaboración propia. Con base en las Historias de usuario, versión 3, y en el tablero v2.

Referencias

Boehm, B., & Turner, R. (2003). *Balancing agility and discipline: A guide for the perplexed*. Addison-Wesley.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The unified software development process*. Addison-Wesley.

Kruchten, P. (2003). *The Rational Unified Process: An introduction* (3.ª ed.). Addison-Wesley.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. https://scrumguides.org/scrum-guide.html